第三章复习大纲：进程与虚拟化

以下是根据您提供的文档（特别是文档4）整理的分布式系统第三章“进程与虚拟化”的复习内容。我将课件中的英文术语和描述翻译成中文，确保全面覆盖，重点内容详细解释并举例，非重点部分关键词概括。同时，我标记了重点内容在文档中的位置（参考文档4中的幻灯片编号，如“第2页”等），并嵌入相关图片以增强理解。回复结构包括概述、分节详解、重点总结和例子。

第三章概述

第三章主要讲解分布式系统中进程与线程的管理、虚拟化技术、客户-服务器模型以及代码迁移。这些内容是分布式系统的基础，涉及资源管理、透明性和可扩展性。重点包括：多线程的优势、虚拟化类型、服务器状态管理以及代码迁移模型。（基于文档4第2-3页）

---

§3.1 进程与线程

位置：文档4第2-4页

关键词概括：进程定义、线程定义、多线程作用、线程级并行度（TLP）。

重点内容：进程与线程的区别、多线程在分布式系统中的优势。

- **进程（Process）**：是包含多个线程的软件处理器，线程在进程上下文中执行。进程上下文包括线程上下文、MMU寄存器值等。
- **线程（Thread）**：是最小的可执行指令序列的软件处理器，线程上下文包括处理器上下文（如栈指针、程序计数器）和状态。
- **为什么在分布式系统中使用线程而不是进程？**（文档2问题16）
- 线程更轻量级：创建和销毁线程的开销远低于进程，避免频繁的进程上下文切换。
- 提高响应性和性能：多线程允许并发处理多个请求，例如Web浏览器使用多线程同时下载文本和图像，避免阻塞（例子：当一個线程等待I/O时，其他线程可继续执行）。
- 经济高效：更好地利用多核处理器，实现并行执行。
- **线程级并行度（TLP）**：衡量线程并发执行的指标，公式为加权平均线程数。TLP值在1.5-2.5之间表示线程主要用于逻辑组织（如浏览器）。
- **服务器构建方式**：多线程模型优于单线程和有限状态机，因为它简化了控制流，允许使用阻塞调用。

  图片说明：多线程服务器允许并行处理请求，提高性能。

---

§3.2 虚拟化

位置：文档4第5-7页

关键词概括：虚拟化作用、虚拟化类型（进程VM、原生VMM、主机VMM）、指令执行、虚拟化条件。

重点内容：虚拟化技术的分类和作用，以及其在分布式系统中的应用（如云计算）。

- **虚拟化的主要作用**（文档2问题17）：
- 应用程序适配：解决硬件与软件兼容性问题。
- 资源需求多样化：为不同应用提供定制环境（例如，游戏需要高性能GPU，而Web服务需要高内存）。

- 可移植性和安全性：虚拟机隔离故障和攻击，如病毒不会影响硬件。
- **虚拟化技术类型** ：
- **进程虚拟机（Process VM）** ：如JVM，运行在OS之上，解释执行指令集，实现跨平台兼容。
- **原生虚拟机监控器（Native VMM）** ：直接运行在硬件上，允许多样化Guest OS，用于服务器虚拟化（例如VMware ESXi）。
- **主机虚拟机监控器（Hosted VMM）** ：利用Host OS能力简化管理，用于桌面虚拟化（例如VirtualBox）。

  图片说明：虚拟化通过抽象接口实现资源隔离。
- **指令执行与性能** ：现代VM性能接近物理机，大部分指令直接被硬件执行。关键概念包括特权指令（仅OS可执行）和敏感指令（影响系统配置），虚拟化要求所有敏感指令都是特权指令（X86通过指令模拟解决）。
- **分布式系统中的应用** ：云计算（IaaS、PaaS、SaaS）基于虚拟化实现资源池化。例如，Amazon EC2使用原生VMM提供虚拟机服务。

---

§3.3 客户-服务器

位置：文档4第8-15页

关键词概括 ：客户端角色、服务器类型（迭代vs并发、有状态vs无状态）、服务器发现、服务器集群。

重点内容 ：服务器状态管理（有状态vs无状态）、服务器发现方法、集群架构（文档2问题18、22）。

- **客户端（Client）** ：提供用户与远程服务器交互的手段，例如网络化用户接口（如X Window系统）。客户端实现透明性（如通过存根处理RPC）。
- **服务器类型** ：
- **迭代服务器（Iterative Server）** ：单进程顺序处理请求，简单但响应慢（例如，简单HTTP服务器）。
- **并发服务器（Concurrent Server）** ：为每个请求创建子进程/线程，并行处理，提高响应速度（例如，Apache Web服务器）。
- **有状态服务器（Stateful Server）** ：服务器保存客户端状态，减少请求大小，但可靠性差（故障时状态丢失）。例子：在线购物车，服务器记录用户添加的商品。
- **无状态服务器（Stateless Server）** ：每个请求自包含，故障恢复简单，但请求需携带更多信息（例如RESTful API）。
- **服务器发现方法**（文档2问题22）：
- **基于目录服务器** ：客户端查询目录服务获取服务器位置，灵活适用于动态环境。
- **基于超级服务器（Superserver）** ：如inetd，监听多个端口，按需启动服务器进程，资源高效。

  图片说明：服务器发现方法影响系统可扩展性。
- **服务器集群** ：提高可扩展性和可靠性。
- **三层架构** ：前端负载均衡器、中间应用服务器、后端数据库。使用TCP移交（TCP handoff）避免单点瓶颈。
- **广域集群** ：如云数据中心，基于DNS分发请求（例如，CDN根据用户位置选择最近服务器）。

图片说明：集群通过负载均衡和TCP移交优化性能。

---

§3.4 计算迁移

位置：文档4第16-18页

关键词概括 ：代码迁移目的、迁移模型（弱迁移vs强迁移）、资源绑定、虚拟机迁移。

重点内容 ：代码迁移的模型和资源处理（文档2问题19、20）。

- **代码迁移的目的** ：
- 负载均衡：将代码从高负载节点迁移到低负载节点。
- 减少通信：将计算移动到数据所在位置（例如，MapReduce将Map任务推送到数据节点）。
- 动态配置：按需安装协议或库。
- **代码迁移模型** （文档2问题19）：
- **弱迁移（Weak Mobility）** ：仅迁移代码段（如JSP），执行从预定义点开始。
- **强迁移（Strong Mobility）** ：迁移代码和执行状态（如线程上下文），实现无缝恢复。
- **发起方式** ：发送方发起（上传代码到服务器）或接收方发起（客户端拉取代码）。

  图片说明：迁移模型影响灵活性和性能。
- **资源绑定问题** ：迁移时需处理进程与本地资源的绑定（如打印机）。
- **绑定类型** ：按标识符（如URL）、按值（复制资源）、按类型（如打印机）。
- **处理动作** ：GR（创建全局引用）、MV（移动资源）、CP（复制值）、RB（重新绑定到本地资源）。

  图片说明：资源绑定决策影响迁移可行性。
- **虚拟机迁移** （文档2问题20）：
- **静态迁移** ：停止VM，迁移内存后重启，服务中断较长。
- **动态迁移（Live Migration）** ：逐步推送内存页面，服务中断短（几秒），例如VMware vMotion用于云环境负载均衡。

重点总结

- **核心概念** ：进程与线程的轻量级优势、虚拟化类型（进程VM、VMM）、服务器状态管理（有状态vs无状态）、代码迁移模型（弱/强迁移）。
- **考试重点** ：
- 为什么用线程？——多线程提高响应性和并行度（文档2问题16）。
- 虚拟化方式？——进程VM、原生VMM、主机VMM的区别（文档2问题17）。
- 服务器状态？——有状态服务器效率高但可靠性差，无状态服务器易于恢复（文档2问题18）。
- 代码迁移？——弱迁移只传代码，强迁移传代码和状态（文档2问题19）。
- **实际例子** ：Web浏览器多线程、云计算虚拟化、TCP移交在集群中的应用。
- **复习建议** ：结合文档3试题练习，如选择题涉及体系架构，简答题关注迁移和虚拟化细节。

此大纲覆盖第三章全部内容，重点突出，例子详细。如果您需要特定部分的深入解释或练习题分析，请随时告知！